ChatGPT

MilkDrop Preset File Format and Sections

A MilkDrop preset (the `.milk` file) is essentially a script that defines how the visualization behaves. It contains several sections of *equations* that MilkDrop evaluates continuously to render graphics in sync with audio. In MilkDrop 2 (and its successors), a preset is composed of four main types of scriptable sections: **per-frame equations**, **per-pixel equations**, **custom shapes**, and **custom waves** [1]. Each section plays a distinct role:

- **Per-frame code:** Runs once *each frame* (typically 30–60 times per second) and updates global parameters like zoom level, rotation, motion speed, waveform color, etc. [2]. This is where you use time variables, audio FFT data, and math functions to make the scene evolve over time and react to music.
- **Per-pixel code:** Also known as *per-vertex* code, it runs across a grid of sample points on the screen every frame (e.g. a 32×24 grid by default) rather than literally every pixel [3]. It modifies how the image is warped or transformed at different screen locations, based on spatial coordinates.
- **Custom shapes:** Definitions for drawn geometric shapes (like polygons) that can appear on top of the visualization. You can have several shapes per preset (4 in MilkDrop 1.x; up to 5 in MilkDrop 2; and 16 in MilkDrop 3 [4]), each with its own properties and optional per-frame code for animation.
- **Custom waves:** Definitions for additional waveform or spectrum visualizations. These are like programmable oscilloscopes – they plot audio data (waveform or FFT) in custom ways. Each custom wave can have a per-frame script and a per-point script to precisely control its appearance.

File Syntax: A `.milk` file is a plain text file. It isn't formatted in a human-readable markup like JSON or XML; instead, it contains key-value pairs and blocks of code. For example, you'll find lines for static parameters (e.g., `wave_r=1.0`, `wave_mode=5`, `shape1_enabled=1`, etc.) and sections where code is written as semicolon-separated expressions (similar to writing mini C/JavaScript expressions). In the Winamp MilkDrop editor, the preset is divided into menus for *Wave*, *Motion*, *Shapes*, etc., but in the file these all live together as text. Key sections include:

- **Preset constants:** e.g. title, rating, and various default sliders (wave modes, colors, etc.).
- **Per-frame equation block:** one or more lines of expressions that execute every frame.
- **Per-pixel (vertex) equation block:** expressions that execute for each grid point (affecting spatial warping).
- **Custom shape blocks:** for each shape, a set of parameters (position, sides, colors, etc.) and an optional per-frame script block.
- **Custom wave blocks:** for each wave, parameters (number of samples, display style, etc.), plus optional per-frame and per-point script blocks.

MilkDrop 2 and later added an option to include **GPU shader code** (HLSL) in the preset as well. If present, shader code appears in special sections (labeled `warp` and `comp` in the file or editor). We'll discuss shaders further below.

Per-Frame Equations (Frame Logic)

The **per-frame** code drives the high-level animation and responds to the music on a frame-by-frame basis. This code is executed once for each video frame. Within it, you can adjust global visualization parameters and any custom variables. For example, you might modulate the zoom factor, rotation angle, waveform opacity, or color components here. The code looks like a series of simple expressions, e.g.:

```
wave_r = wave_r + 0.5*sin(time*1.13);
wave_g = wave_g + 0.5*sin(time*1.23);
wave_b = wave_b + 0.5*sin(time*1.33);
zoom = 1 + 0.03*bass;
```

Each frame, MilkDrop evaluates these in order. You have access to a set of built-in variables and functions. Important inputs include the special time variables and audio spectrum variables:

- `time`: the current playback time in seconds (continuous) [5].
- `frame`: the frame counter (increments each frame) [6].
- `fps`: the current frame rate (frames per second) [7].
- `progress`: how far we are into the current preset's lifespan (0 to 1 from start to end of preset; if presets aren't on a timer this may just run 0→1 slowly or be static) [8].

And critically, **audio-reactive variables**: MilkDrop performs an FFT on the audio each frame, dividing it (stereo) into frequency bands. It provides: `bass`, `mid`, `treb` for low, mid, and high frequency amplitude levels (roughly normalized so ~1.0 is average level) and `bass_att`, `mid_att`, `treb_att` which are *attenuated* (smoothed) versions that don't jump as sharply [9]. A value >1.0 indicates above-average energy in that band (e.g. a bass drop might push `bass` to 2 or more), while values around 0.5 or 0.7 would be relatively quiet. These are the primary hooks for making visuals react to music. For instance, a common technique is to tie the zoom or rotation to the bass level (zoom out on kick drum hits) or to change colors when treble is high, etc.

The per-frame code can use standard arithmetic, logical, and trigonometric functions. MilkDrop's scripting language is simple but fairly powerful – it's derived from Nullsoft's **EEL** (the same expression evaluator from Winamp's AVS plugin) [10] [11]. You have the usual operators (`+ - * /` for arithmetic, `=` for assignment, and even bitwise ops) and many math functions: `sin(), cos(), tan(), asin(), acos(), atan()`, exponentiation (`pow`, `sqrt`), logs, `abs`, as well as some handy ones like `min, max, sign, if(cond, trueVal, falseVal)` and boolean helpers `above(a,b)`/`below(a,b)` to return 1 or 0 based on a comparison [12] [13]. Randomness is available via `rand(N)` which gives a random integer 0 to N-1 [14]. There is no explicit looping construct (no `for` or `while`), but since the code runs every frame, you often achieve iterative effects by using frame counters or oscillatory functions.

What can you do in per-frame? A lot: You can continuously rotate the image (`rot` variable), zoom in/out (`zoom`), shift it (`dx`, `dy` for drift), adjust distortion (`warp` amount), or alter the drawn waveform's attributes (`wave_r`, `wave_g`, `wave_b`, `wave_a` for waveform color and alpha, etc.). You can also define your own temporary variables on the fly. If you write `myVar = sin(time);` then `myVar` exists for this

frame (and beyond, if you declared it in the init section – more on variable persistence later). Per-frame is also where you'd implement conditional logic, e.g. flash the screen white on a beat: `alb = if(bass_att > 2.5, 1, alb*0.9);` (MilkDrop has an `alphablend` or brightness factor `alb` for the whole screen – this is just an illustrative example). In short, per-frame code handles *temporal effects* (how things change over time globally and in response to music) [15].

Because it executes so often, per-frame code needs to be efficient (and it generally is just a few lines of algebra). This code sets up values that the subsequent stages (per-pixel and shapes/waves) will use for finer detail.

Per-Pixel (Per-Vertex) Equations (Spatial Warping)

The **per-pixel** equations (MilkDrop 1.x called them per-vertex) run after the per-frame code each frame. Despite the name, they are *not* executed for every single pixel on your screen (that would be too slow in software). Instead, MilkDrop divides the screen into a grid (by default 32×24 = 768 points) and runs the per-pixel script at each grid point [3]. It then interpolates the results to apply to all pixels in between. Essentially, it creates a low-poly mesh and warps it according to your equations, which gives a smooth continuous warp across the full image. The grid density can be adjusted by the user; a higher mesh (say 64×48 or more) yields more accurate but slower per-pixel effects, whereas a low mesh (e.g. 16×12) is faster but can show artifacts. This is a classic performance trade-off, and MilkDrop even allowed dynamic mesh-size adjustments for speed.

Purpose of per-pixel code: It lets you vary certain transformation parameters across the *spatial* dimensions of the screen, instead of applying one value globally. For example, maybe you want the edges of the screen to twist more than the center, or create a fisheye lens effect, or have the background zoom in a spiral pattern. Per-frame code alone can't do that because it sets one value for the whole frame. Per-pixel code, by contrast, can say "at this x,y coordinate, modify the zoom by *this* amount, but over here, modify it by a different amount."

MilkDrop provides special read-only variables in per-pixel context that tell you the position: `x` and `y` (normalized screen coordinates, 0 to 1) of the point being processed, `rad` (the radial distance of this point from the center of the screen), and `ang` (the angle of the point in polar coordinates) [16] [17]. Using these, you can craft spatial effects. A classic example: *perspective warp*. Normally if you zoom an image, it's uniform and gives no 3D depth feeling. But if you reduce zoom in the center and increase it toward edges, the result looks like a 3D bulge. You can do this with one line in per-pixel: `zoom = zoom + rad*0.1;` which adds 10% extra zoom at the edges (`rad` is 0 at center, ~1 at corners) [18]. MilkDrop's authoring guide illustrates this exact snippet and its effect [18].

What can you control per-pixel? The key *writable* variables here include most of the transformation parameters: `zoom`, `zoomexp` (zoom exponent for curvature), `rot` (rotation), `warp` (the warping strength for the feedback effect), `cx`/`cy` (the center of zoom/rotation), `dx`/`dy` (translational movement), `sx`/`sy` (stretch/skew on X and Y) [19] [20]. By default, these were all set by the per-frame stage as single values; in per-pixel, you are essentially adding offsets or altering them based on position. You do *not* directly set color here (colors are handled by waves and shapes or global tint; per-pixel is mainly geometry).

One thing to note: per-pixel code runs on the *previous frame's image* in MilkDrop's pipeline. MilkDrop's visuals use feedback: each frame, the last frame's image is warped and then new waveforms/shapes are drawn on top. The per-pixel equations control that warping of the last frame (this is called the "motion" or "warp" stage). So, if per-frame set `rot = 0.1`, that means "rotate the whole image 0.1 radians this frame." If per-pixel then does something like `rot = rot + 0.1*rad`, now at the center `rot` is 0.1 but at edges `rot` might be 0.1 + 0.1*1 = 0.2 radians. The outer parts twist twice as much as center, causing a spiral streaking effect. Because of the feedback nature, these warps accumulate frame to frame, often producing intricate patterns.

Interpolation and quality: As mentioned, MilkDrop interpolates between the grid points after computing per-pixel code. This means your per-pixel formulas are effectively smooth across the screen, and you rarely see the grid unless using a very low mesh size or extremely high-frequency changes. The default 32×24 grid was chosen as a good quality-performance balance [3]. If you push mesh higher (in config, up to 192×144 max), the per-pixel code executes many more times per frame (impacting CPU) but can capture finer detail. Modern systems handle quite high mesh sizes easily, and ProjectM even allows adjusting this if needed.

Access to audio in per-pixel: Interestingly, you *can* use the audio variables (`bass`, etc.) inside per-pixel code as well, but note that those are constant across the screen (they don't vary with `x` or `y`). So if you reference `bass` in per-pixel, you'll get the same value at every vertex – meaning effectively you'd be applying some bass-dependent offset uniformly. In practice, that's usually done in per-frame instead (for efficiency, you wouldn't need to recompute it at every grid point). However, you might use audio in a conditional way in per-pixel, e.g., only warp outer pixels if bass is above a threshold (making the screen shake on bass hits at the edges). This is a niche use; generally per-pixel focuses on spatial coordinates.

One must also be careful with custom variables: any variable you create inside per-pixel code (that isn't a built-in like `x,y,rad`) is essentially local to each vertex calculation. It won't automatically carry over from one point to the next, or to the next frame, unless you explicitly use the bridging mechanism we'll talk about (the `q` variables). In essence, per-pixel code should be written as a pure function of position and the current frame's context. It's like a vertex shader in modern GPU terms – no persistent state per vertex from last frame (unless you coded something into a global).

Waveforms and Custom Waves

MilkDrop always had the concept of drawing the audio waveform (or spectrum) over the visuals. In classic MilkDrop (1.x), the user could choose a **waveform mode** (e.g., a single oscilloscope line, double mirrored waves, a filled oscilloscope, a dotted line, etc.) using the preset settings [21] [22]. There are 8 built-in waveform styles selectable via `wave_mode 0` – `7` [21]. Along with that, there are parameters to control the wave's color (`wave_r, wave_g, wave_b`), transparency (`wave_a`), position (`wave_x, wave_y` shift), and style toggles like `wave_thick` (draw a thicker line) [23], `wave_additive` (use add blend so bright parts intensify colors) [24], `wave_brighten` (auto-brighten the wave) [24], and `wave_dots` (draw as points instead of continuous line) [25]. There's even the mysterious `wave_mystery` parameter whose exact function wasn't officially documented (hence the name) [26] – it's a quirky parameter that affects the shape of certain waveforms in non-obvious ways.

These **built-in waveforms** are drawn by MilkDrop automatically each frame, and you can animate them by changing the above parameters via per-frame code. For example, you could oscillate `wave_r/wave_g/`

`wave_b` to cycle the wave's color [27], or increase `wave_y` to move the waveform vertically. Many presets use the built-in waveform (it's the wiggly line often seen in the center). However, the built-in options, while useful, are limited to a few shapes (line, doubled line, circle, etc.) and behaviors.

Custom waves extend this by giving the preset author full control of how to draw the wave or spectrum. As of MilkDrop 1.04, custom waves became available, up to 4 in a preset (MilkDrop 2.x also defaulted to 4; MilkDrop 3 increases to 16) [4]. A custom wave lets you essentially plot the audio data in any shape or pattern you want, using scripting.

Each custom wave has its own set of parameters and code sections:

- **Wave attributes (static):** In the preset file you'll have fields like `waveN_enabled` (turn it on/off), `waveN_samples` (how many sample points, up to 512) [28], `waveN_sep` (maybe a stereo separation flag), `waveN_bSpectrum` (whether to use frequency spectrum instead of waveform amplitude), and base color fields (`waveN_r,g,b,a`). Not all are directly documented in the snippet above, but basically the MilkDrop menu allowed you to set these for each custom wave. For instance, you could choose to sample 512 points of the *spectrum* (instead of waveform) and draw them.
- **Per-frame code (custom wave):** Much like the main per-frame, each custom wave can have a small code block that runs once per frame [29] [30]. In it, you typically adjust the wave's color or other properties over time. For example, you might make the wave's opacity pulse with the music, or change the number of samples dynamically (`samples` is writable in code [28], so you could use fewer points when music is quiet, etc.). The variables accessible here include time, frame, the audio variables (`bass` etc.) [31] [32], and also a special set of *Q and T variables* (discussed later) that carry info from the main script. The wave's per-frame code cannot directly set its points – that's the job of the per-point code – but it can set overall settings and prepare values.
- **Per-point (per-vertex) code:** This is the powerful part. For each of the `N` sample points in the wave, MilkDrop runs the per-point code to determine where that point is drawn and what color it has [33] [34]. By default, if you *don't* provide per-point code, MilkDrop would draw the samples as a standard wave (left to right, center vertical position, connecting points). But with code, you can plot them arbitrarily. The per-point code has its own variables:
- `sample` (a 0..1 value indicating the position along the list of samples, i.e. 0 for first point, 1 for last) [35].
- `value1` and `value2` which are the audio data at that point – if using waveform, these are left/right channel amplitude at that sample; if using spectrum, they are magnitudes for that frequency band [36].
- You can set `x` **and** `y` (0..1 screen coordinates) for each point to plot it wherever you want [34]. For instance, you could map `x = 0.5 + 0.4*sin(sample*6.28)` and `y = 0.5 + 0.4*cos(sample*6.28)` to make the waveform draw in a circle instead of a line. Or create a spiral, or a vertical bar – anything you can express mathematically.
- You can also set the color per point: `r, g, b, a` (these default to the base color from the per-frame section, but you can override per point) [37]. So you could make the waveform gradient or flash certain parts.

In essence, custom wave per-point code is like writing a little program to draw a polyline or point-cloud, where the "data" you're plotting is the music. It's extremely powerful – many flashy presets draw complex patterns (swirls, circles, expanding rings) that are actually just the audio waveform drawn in a fancy path.

Because custom waves are so flexible, they effectively supersede the built-in waveform in advanced presets. MilkDrop 1.x only had built-ins, but from 1.04 onward authors often disabled the regular wave (`wave_mode=0` meaning off) and used a custom wave for cooler effects. For example, drawing spectrum bars in a circle: you can take the 64 FFT bands and place them as radial lines around a circle by computing coordinates from the `sample` index – something impossible to do with the old modes.

One important note: custom waves can choose to use the waveform or spectrum. Typically a toggle (in the MilkDrop editor UI it's "Spectrum" on/off for custom wave). If spectrum mode is on, then MilkDrop will feed frequency data into `value1/2` instead of time-domain wave samples. So you might then use `sample` (0..1) to correspond to frequency from low to high and `value1` as the magnitude. Many presets do this to draw an equalizer-like shape.

Custom Shapes

Custom shapes were another big addition in MilkDrop 1.04+. They let preset authors draw geometric shapes (triangles, circles, stars, etc.) on screen that can move and change each frame. Prior to that, MilkDrop had no easy way to render arbitrary polygons or images; you were limited to the waveform and the smearing of the previous frame. Custom shapes opened up a huge range of visual possibilities (from simple pulsing circles to complex tiling patterns and even sprite-like use cases).

Basics: A custom shape is essentially a polygon defined by a number of sides (between 3 and 100 in classic MilkDrop) [38] [39]. If you set sides = 100, it's basically a circle; if sides = 5, a pentagon; 3 is a triangle; etc. You can control the shape's **position** (`x`, `y` on screen, where (0.5,0.5) is center), its **size** (`rad` = radius), and its **rotation angle** (`ang`) [40] [41]. You also specify **colors**: MilkDrop uses a two-color gradient for filling the shape – you set inner color (`r,g,b,a`) and outer color (`r2,g2,b2,a2`) [42] [43], and it will color the shape smoothly from center (color1) to edge (color2). Additionally, there's a **border** (outline) color `border_r,g,b,a` if you want an outline drawn [44]. You can toggle whether the shape is **additive** (`additive=1` means it adds its color onto the background like light; 0 means it alpha blends normally) [45] [46]. And you can choose a **thick** outline option which overdraws the border a few times to make it brighter/bolder [47] [45].

A very special option is `textured`: if `textured=1`, then the shape is not a solid color – instead, it is filled with the *previous frame's image*, mapped onto it [48]. This effectively lets you create feedback mirrors and tunnels. For example, the "Geiss – Feedback (feedback tunnel)" preset uses a textured shape that takes the last frame and maps it onto a rotating square, creating a hall-of-mirrors effect [49]. When `textured=1`, the `tex_zoom` and `tex_ang` parameters control how that texture is scaled and rotated inside the shape [50] [51]. So you could take the last frame, rotate it 45 degrees and shrink it inside the polygon, etc., achieving recursive visuals.

Like waves, each custom shape in the preset can have a **per-frame code block** to animate its properties [52]. In that code, the shape-specific variables (x, y, rad, ang, r, g, b, etc. for that shape) are all available and writable. For instance, you might do `ang = ang + 0.1;` so the shape spins continuously, or `rad = 0.2 + 0.1*sin(time*3);` to pulse the radius. The shape's code also sees the audio and time variables, so you can make it react (e.g. change size with bass). Essentially, the shape per-frame equations work just like the main per-frame, but only affect that shape's parameters [53]. MilkDrop will execute the shape's per-frame code once each frame (per shape), then draw the shape.

A powerful feature is **instancing** of shapes: Each shape entry can be drawn multiple times per frame as a series of instances. There are two special variables for this: `num_inst` (how many instances total) and `instance` (the index of the current instance, from 0 to num_inst-1) [39] [54]. MilkDrop will loop through the shape's per-frame code `num_inst` times, each time setting the `instance` number. Your code can use `instance` to, say, offset the position or angle for each copy. This means one shape definition can actually produce dozens or even hundreds of graphical elements on screen. For example, you could set `num_inst=100` and then in code do `ang = instance * 0.0628; x = 0.5 + 0.3*cos(instance*0.0628); y = 0.5 + 0.3*sin(instance*0.0628);` to place 100 small shapes in a circle pattern. Each loop could also use time or audio, e.g. color each instance differently or scale it based on `bass`. **Up to 1024 instances** are allowed for one shape [55] [56] (though that many might be slow). This instancing effectively provides a *loop* inside the preset – one of the only ways to get a loop-like behavior since the expression language itself has no loops. MilkDrop 1.x/2 allowed 4 shape definitions, each with up to 1024 instances, which is plenty. (MilkDrop 3 expands to 16 shapes, still up to 1024 inst each, and even raises max sides to 500 for very complex polygons [57].)

To summarize, custom shapes are like independent overlay graphics you script. They can be as simple as a static border or as complex as dozens of moving pieces that react to the music. **Custom waves and shapes greatly enhance the visual variety**: MilkDrop's built-in warping + waveform is already dynamic, but adding scripted shapes and waves means you can display text-like patterns, flashing geometric grids, rotating 3D-like objects, etc., all integrated with the beat of the music.

Frame-by-Frame Execution and Variable Scoping

With all these pieces (per-frame, per-pixel, shape, wave, etc.), it's important to understand *when* each runs and how they interact each frame. The **execution model** of MilkDrop is deterministic and runs in a fixed sequence every frame:

1. **Preset Init (once on load):** When a preset is loaded, there is an optional "init" code section that runs. Here you might initialize variables that you want to persist for the life of the preset (e.g., set up random offsets, or starting values for counters). This is not always explicitly present in presets (if omitted, it's just empty), but it's where you'd put one-time setup. Variables set here will carry into the first frame's computations.
2. **Per-frame equations (preset):** At the start of each frame, MilkDrop evaluates the preset's per-frame code [2]. This updates global variables for this frame based on last frame's state and current inputs (time/audio). For example, you compute new `zoom`, `rot`, `wave_r` here.
3. **Per-pixel (vertex) equations (preset):** Immediately after per-frame, MilkDrop takes the values from per-frame and runs the per-pixel code across the mesh [58]. At each vertex, it may modify `zoom`, `rot`, etc. according to position. These adjustments apply to the warp of the *previous frame's texture*. When this is done, MilkDrop uses those transformed coordinates to sample the last frame's image and draw the warped background for the current frame [59]. In MilkDrop 2, if a **warp shader** is present, it executes *instead of* the CPU per-pixel script (more on that soon). Conceptually, both do the same job: warping the old frame into the new frame's background.
4. **Custom shapes (per-frame and draw):** Next, for each custom shape that is enabled, MilkDrop runs that shape's per-frame code. If `num_inst` for that shape is greater than 1, it will loop through the code that many times, updating the shape's variables each iteration with `instance` number [55] [54]. Each iteration is then immediately drawn as a separate shape instance on the screen. So

essentially, shape drawing happens during this step. MilkDrop goes shape1, runs its code (loop), draws all instances of shape1; then shape2, etc. The shapes are drawn on top of the already-warped background. They are typically drawn with blending (alpha or add) onto the scene.

5. **Custom waves (per-frame and per-point):** After shapes, for each custom wave that's enabled, MilkDrop runs the wave's per-frame code [29] . Then it retrieves the current audio data (waveform or spectrum) for the number of samples specified. It then runs the wave's per-point code for each sample [33] , plotting the points (or lines between them) as specified. This renders the custom waves on top of everything (they often appear as bright lines or patterns). If there is no per-point code, MilkDrop just draws the points in a default manner (e.g. connected line).
6. **Built-in waveform and final composite:** If the preset still has the built-in waveform enabled (`wave_mode` not 0), it will draw that now using the current `wave_r,g,b,a,etc.` parameters [21] [26] . This always draws on top of shapes (unless those were additive with full brightness). In practice, many modern presets either use custom waves or set the built-in wave to a subtle mode. But if used, it's drawn here.
7. **Post-processing shaders (MilkDrop 2+):** In MilkDrop 2/3, there is also a **composite shader** stage after all drawing [60] . If the preset includes a composite pixel shader, it runs now, operating on the whole final frame image before it's presented. Composite shaders can do things like full-screen color effects, blur, fish-eye lens, etc., beyond what the basic pipeline does. If no comp shader is present, the image goes out as-is.
8. **Display / feedback loop:** The frame is displayed. Then, internally, MilkDrop takes this frame and treats it as the "previous frame" for the next iteration. So the next cycle, the image we just drew will be the one that gets warped by per-pixel or shaders.

This sequence happens typically 60 times a second (or whatever frame rate). Meanwhile, audio data is streaming in and FFT is giving new bass/mid/treb values each frame.

Variable scoping and persistence: Because there are multiple "contexts" (preset frame vs. per-pixel vs. shape vs. wave), MilkDrop organizes variables into what the authoring guide calls *variable pools* [61] . The rules can be a bit confusing, but in summary:

- Variables declared at the **preset level** (either in init or in per-frame code) persist from frame to frame [62] [63] . For example, if in per-frame you do `mycount = mycount + 1;` and you initialized `mycount` in init to 0, then `mycount` will increment each frame – it remembers its value. All preset-level standard variables (like `zoom`, `rot`, etc., and your custom ones) carry over frame to frame *unless* you deliberately reset them. This means you can have accumulating effects or state machines.
- Variables in **per-pixel code** do *not* persist in the same way. Each time the per-pixel equations run, it's as if starting fresh for that vertex. Also, you cannot see normal preset variables directly in per-pixel scope (the per-pixel code is executed in a separate context for each vertex). For example, if you set `foo=5;` in per-frame, and then try to use `foo` in per-pixel, it won't automatically have 5. The preset authoring guide explicitly warns that user-defined vars don't carry from per-frame to per-vertex – you'll "not get the correct value" [64] . The solution is to use the special bridging variables Q1–Q32 (coming next).
- Variables in **shape per-frame code** and **wave per-frame code** also do not see the preset's normal variables unless bridged. Each shape or wave has its own context. However, shapes/waves *can* read the global audio and time directly (those are global read-only) [31] , and they can use their own persistent variables if needed (a variable you declare in a shape's per-frame will persist to that shape's next frame, similar to preset-level persistence).

- Variables in **wave per-point code** are ephemeral to each point. Generally, you treat per-point a bit functionally – it computes output for a point without memory of the last point (except if you explicitly carry something via Q or T arrays between points, e.g. to do smoothing as the guide suggests) [65] .

To share data between these stages, MilkDrop provides **Q-vars** and **T-vars** as pipelines:

Q1–Q32: Think of the Q variables as a 32-slot array of floats that each stage passes along [66] [67] . The life cycle is:

- You can set Q-vars in the **preset init** code (e.g., set `q1=0` initially, etc.). Those initial values will then be available in subsequent frames.
- At the start of each frame, MilkDrop resets `q1..q32` to whatever they were left as at the end of the *previous frame's* preset per-frame code [68] [69] . Then it runs the preset per-frame code, which can modify q's further. (In practice, this means you can use q's as a carry-over between frames *if you update them every frame*, because if you don't change a particular q, it will keep the last value from previous frame's end.)
- Right after preset per-frame, when MilkDrop enters per-pixel, it makes sure the `q` values are those from end of per-frame. The per-pixel code can *read* q1–q32 (they are available to use), but per-pixel code typically shouldn't modify q's (if it does, it only affects that vertex's copy – not globally).
- More importantly, when MilkDrop calls **shape per-frame** and **wave per-frame** code, it *copies in* the q1–q32 values from the end of preset per-frame as well [68] . So the first line in a shape's per-frame sees q's equal to whatever the main per-frame left them as. This is how information "flows downhill" into shapes and waves. For example, if your preset per-frame computed a complex pattern and stored some results in q1–q4, your shape code can now use those q1–q4 values.
- Inside shape/wave per-frame, you can also modify q variables if needed. However, these changes won't go back and affect the preset or other shapes *during that frame*. They only matter in one specific case: **wave per-point code**. When a custom wave's per-frame code finishes, whatever q's it set are then used as the starting q values for that wave's per-point iterations [70] . Furthermore, within the per-point loop, if you update a q (say you accumulate something across points), the updated q persists to the next point in that same frame [70] . This allows techniques like smoothing: you could set `q1 = 0.8*q1 + 0.2*value1;` for each point to low-pass filter the waveform. At the end of the wave's points, those q changes are thrown away (they don't persist to next frame unless you also stash them somewhere like a T or re-output to itself in wave per-frame every time).
- Q's do **not persist frame-to-frame automatically**; they reset each frame to the preset per-frame output as described [69] . If you want something persistent, you either use a normal custom variable or you continually feed a value into q from a persistent variable. Essentially, q's are for bridging between different code contexts in *the same frame* [68] [71] .

T1–T8: These are similar but local to shapes and waves [72] . T-vars exist because sometimes you need to bridge between *init* and *per-frame* inside a shape/wave, or between wave per-frame and wave per-point, without interfering with the preset's Q's. The flow is simpler: for a given custom shape or wave, T1–T8 can carry values from that shape's init code to its per-frame, and from shape per-frame to each instance iteration (for waves: wave init → wave per-frame → wave per-point) [73] . They basically solve similar scope gaps but are isolated for each shape/wave. Many preset authors might not need T's, but they are there for more complex state passing.

Persistence of custom variables: The general rule is *custom variables persist within the context they're declared in, but not across context switches*. So:

- If you declare a variable in **preset per-frame** (or init), it stays around next frame in preset per-frame (so long as the preset stays loaded) [62] [63]. For example, `float beatCounter;` in init and then using it in per-frame to count beats – it will remember its value.
- If you declare a var in **per-pixel**, each frame it starts fresh (and even each vertex calculation it starts at default). The MilkDrop guide suggests treating per-vertex user vars as scratch only [74].
- In **shape per-frame**, a static var would persist next frame for that shape. But shape per-frame is called after preset per-frame each time; if you need something from preset scope, you'd use Q or just refer to a global variable (since preset variables *are* in scope inside shape code, but their values might be stale if not updated to Q – actually to clarify: preset's global variables like `zoom` might not mean much inside shape code because the shape doesn't use `zoom`. However, you could technically read a preset variable in shape code if it's not overridden. Usually one uses q's for clarity).
- If you declare a var inside the shape's per-frame code itself, it will persist next frame (similar to how preset per-frame works) because that shape's code is run anew each frame and the variable retains its last value.
- Wave per-point variables reset each frame (unless carried by T or Q as mentioned).

This system is admittedly complex, but the MilkDrop authoring guide provides diagrams and explains: the Q's and T's are the intended method to *transfer* values between the different execution pools [75] [72]. As a preset author, you often do something like: compute an expensive or global calculation in preset per-frame, store it in, say, `q1`; then use `q1` inside your per-pixel or shape code many times (saves recomputation and ensures consistency).

Frame-by-frame flow with an example: Suppose you want to make a shape that always sits at the point of highest bass response on the screen (just a contrived example). You could in per-frame compute the "bass angle": `q1 = ang + bass*1.57;` (say, whatever logic to get an angle from bass). Then in your shape's per-frame, you set `ang = q1;` so the shape's rotation comes from that. The sequence would be: each frame preset sets q1; then shape reads q1. If you didn't use q1 and tried to use some normal variable, it might not update at the right time or be out of sync.

The bottom line: **MilkDrop's execution is a pipeline** – preset frame code → pixel warp → shape code → wave code – and the state flows through global variables and these special arrays. This was all designed to maximize flexibility while keeping performance reasonable (by avoiding truly global variables that update at per-pixel frequency, for instance).

Shader Integration (MilkDrop 2 & 3)

MilkDrop **1.x** did all its rendering with these CPU-calculated equations (plus some fixed-function DirectX tricks). In **MilkDrop 2.0** (2007), Ryan Geiss introduced GPU **pixel shaders** to the mix [76]. This was a big shift because it allowed much more complex pixel-by-pixel computations to be done on the graphics card, enabling effects that would be too slow in the old system, and opening the door to new visual styles.

In MilkDrop 2 (and continuing in MilkDrop 3 and ProjectM), each preset can include up to two shader programs (written in HLSL, shader model 3 targets originally):

- The **Warp shader**: This runs in place of the per-pixel equations (if present). It is a pixel (fragment) shader that takes the *previous frame's texture* as input and outputs a warped version for the current background [60] [59]. In the shader code, you don't manually loop over pixels; the GPU does that. The shader has access to a few inputs like the current UV coordinates, time, and some of the same variables that the CPU code would have had. The MilkDrop authoring guide indicates that in the warp shader, you get variables like `uv` (the coordinate you should sample from the previous frame), and you can manipulate it [77] [78]. Essentially, the warp shader can do arbitrary distortion: you could implement a fish-eye lens, complex swirling, zoom oscillations, etc., with math or even texture lookups that would be hard to approximate with 32×24 grid. If no warp shader is present, MilkDrop falls back to using the per-pixel equations on the CPU as we described earlier.
- The **Composite shader**: This runs after the warp (and after shapes & waves), on the final composed image, right before displaying [60]. It can be used to apply full-screen effects. For example, you could write a composite shader to simulate an old TV (distorting colors via a filter), or do additional blur, bloom, mirror effects, etc. The composite shader can sample the "Main" texture (the result of the warp + drawing) and produce a final color. If no comp shader is given, MilkDrop by default just draws the shapes and waves as normal.

The preset file will have sections for these shaders if used, usually as large blocks of HLSL code. Not all presets use them; many presets remained purely equation-based. But some of the most advanced (and GPU-intensive) visuals come from shader presets.

From a *compatibility* standpoint, these shaders introduced a challenge for open-source reimplementations like ProjectM, because MilkDrop's shaders are written in DirectX/HLSL. ProjectM initially used NVIDIA's Cg toolkit to translate and run these on OpenGL, but Cg is long deprecated [79]. Newer ProjectM versions have built their own HLSL->GLSL translator and support a lot of MilkDrop shader syntax now (with some limitations on very advanced or DirectX-specific shader functions) [80] [81]. This means most presets with shaders will work in ProjectM 4.x by automatically converting the shader code to GLSL behind the scenes. MilkDrop 3 (which is essentially a fork/enhancement of MilkDrop 2) still uses HLSL on Windows; it remains backward-compatible so it retains the same shader system [82].

It's worth noting that MilkDrop's shaders are **pixel shaders only**; there is no user-defined *vertex shader* in presets. The vertex processing (for the mesh, shapes, etc.) is fixed. The pixel shaders were mainly introduced to offload the per-pixel calculation to GPU and to allow new effects that weren't possible with the old fixed pipeline (like multi-pass blur or advanced color effects). In the MilkDrop authoring guide's shader reference, they describe available shader constants and some intrinsic functions (like sampling special textures: MilkDrop provides some built-in noise textures and blur textures for use in shaders to do feedback effects) [83] [84]. Shaders can also get inputs like the `q` variables (MilkDrop passes some of the Q array and other uniforms into the shader so the CPU-side calculations can influence the GPU code). This way, you can still use the familiar MilkDrop variables inside the shader, but you might write more complex math in HLSL than the expression language allows.

Example of shader use: A preset might use the warp shader to create a feedback fractal zoom. The HLSL warp code could take the current pixel's UV, adjust it like `uv = (uv - 0.5) * 0.9 + 0.5 + float2(sin(time), cos(time))*0.1; ret = tex2D(sampler_main, uv);` – which would zoom out

the image around the center and oscillate it with time, sampling the previous frame (`sampler_main` is the last frame's texture) [77] [85]. Doing that on GPU is smooth; on CPU you might not even attempt such continuous zoom each frame. Meanwhile, the composite shader could add a subtle color tint or mirror effect. The possibilities are huge – essentially you could implement any fullscreen pixel transformation akin to a simplified fragment shader from Shadertoy, but within the MilkDrop preset ecosystem.

MilkDrop 3 doesn't change the shader language, but it did raise some limits (like more `q` variables, as mentioned, and more shapes/waves which indirectly means more data for shaders if they use those). It also added new simple waveforms and such, but the core shader concept is the same.

Tools and Resources for Preset Creation

Creating MilkDrop presets involves both coding (or at least tweaking math) and visual experimentation. Over two decades, the community has built up various tools and resources to make this easier. Here are some of the ways you can create or modify presets, from manual editing to automated generation:

Interactive Editors and Runtime Tools

- **MilkDrop's built-in editor (Winamp):** If you run MilkDrop 2 in Winamp, you can press `M` to bring up an on-screen menu to edit the current preset's values and code in real time [86] [87]. You can navigate with arrow keys and edit equations, seeing the effect immediately. It's a bit old-school (text-mode interface) but it works. You'll want to enable *Scroll Lock* to freeze preset switching while you work (so MilkDrop doesn't suddenly switch presets and wipe your changes) [88]. This built-in editor is how many authors in the 2000s made presets – by iteratively adjusting numbers, adding code, and observing. It even has conveniences like copy-paste within the menu and hotkeys (like pressing `I`/`O` to zoom in/out during editing) [89]. The advantage is you get immediate feedback. The downside is it's within Winamp, so not a modern UI, and if Winamp isn't playing audio you don't see much (though you can feed line-in or use a mic for live audio). Still, for learning, it's worthwhile to try: load a preset, open the editor, and tweak a simple variable (say increase `rot` or change a color) to see what happens.

- **ProjectM Standalone & Integrations:** ProjectM, the open-source MilkDrop reimplementation, comes in various forms. There's a **standalone SDL application** (for Windows/macOS/Linux) that can capture audio and run presets [90]. There are also plugins for players like foobar2000, VLC, etc., and even mobile apps. While ProjectM's current releases don't have a polished real-time editor UI, they do usually have a configuration file or console where you can specify a preset to load. For creation, you might edit the `.milk` file in a text editor and reload it in ProjectM to see changes. The benefit of ProjectM is you're not tied to Winamp; you can use any audio source and platform. It's actively maintained – for example, version 4 improved a lot of rendering details to match MilkDrop (instanced shapes, proper wave drawing, noise textures, etc.) [91] [92] – so what you see is very close to Winamp MilkDrop. If you're coding presets, you can keep a ProjectM window open and quickly refresh the preset (some frontends even watch the file for changes to auto-reload, or you can hit next/previous preset to reload it). This is a more manual workflow but effective.

- **MilkDrop 3 (MilkDrop2077):** This is a community-driven enhanced version of MilkDrop 2 that runs as a **portable standalone app on Windows** (no Winamp needed) [93]. MilkDrop 3 (often stylized as

MilkDrop3) by project “milkdrop2077” offers the same preset compatibility and editing as MilkDrop 2, plus *lots* of new features useful for creation. It has a built-in preset mixer where you can layer two presets (the “double-preset” .milk2 format) and even automatically generate mashups by pressing F9 + Space [94] [95]. For a novice user, this is fantastic because you can create something new “without knowing any lines of code” [95] – the program essentially picks two presets and blends their equations for you. MilkDrop3 also added one-key color theme randomization (C key) and a beat-triggered auto preset changer (it can advance to the next preset on a beat drop, as an option) [95] [96]. These features can inspire new presets or variations with minimal effort. It supports up to 16 custom shapes/waves for more complex visuals, and even extends the Q-range to q1–q64 for advanced scripts [97]. Another boon for creators: pressing N in MilkDrop3 shows debug information – presumably values of variables, CPU/GPU usage, etc. – which can help in fine-tuning your preset logic [98]. MilkDrop3 is open source (on GitHub) and is actively updated with community contributions (even some AI assistance in optimizing functions, as noted) [99]. For Windows users, MilkDrop3 is one of the easiest ways to play with presets today: it works with any audio (Spotify, system sound, etc., via loopback capture) and doesn’t require the old Winamp setup [93].

- **Butterchurn (Web):** Butterchurn is a **WebGL/JavaScript** implementation of MilkDrop that runs in a browser [100]. You can visit the Butterchurn Visualizer site or use Webamp (a web version of Winamp) [100] to see MilkDrop presets in your browser. For creation, Butterchurn isn’t an editor per se, but it’s great for quick testing and sharing. You can drag-and-drop preset files into some Butterchurn demos to load them. Under the hood, Butterchurn converts .milk files into a JSON format and even compiles the equations to WebAssembly for performance [101], achieving surprisingly smooth results (the team even got a ~73% performance boost by doing this compilation [101]). If you’re tweaking a preset, you could use Butterchurn for instant feedback: have a local HTML page running Butterchurn, edit your preset in a text editor, and refresh the page. Since it’s web technology, it’s very accessible. Butterchurn is also open source on GitHub [102].

- **VJ Tools (NestDrop, etc.):** There is a tool called **NestDrop** (a separate app that integrates MilkDrop presets into a VJ workflow) which is popular among live visual artists. NestDrop essentially uses MilkDrop presets (via ProjectM under the hood, I believe) to generate visuals for live music, including in dome environments. It provides a nicer UI to trigger presets, mix them, and so forth. While NestDrop is aimed at performance rather than editing, it’s part of the ecosystem making MilkDrop visuals more accessible. Some VJ communities (like VJ Union) share knowledge on using MilkDrop3/NestDrop for live shows. If one’s goal is to create visuals for a concert with minimal coding, a tool like NestDrop might be useful (it allows selecting presets and maybe some parameter tweaking on the fly, without delving into code).

Preset Libraries and Example Content

One of the best ways to learn or to get amazing visuals with little effort is to tap into the **huge library of community presets**. Over nearly 20 years, authors have shared tens of thousands of presets. There are packs like a **73,000+ presets megapack** on the Winamp forums [103], and collections on websites (Milkdrop.co.uk, DeviantArt, etc.), not to mention the presets that come bundled with ProjectM or Winamp. By loading up a variety of presets, you can see what’s possible and even combine ideas.

Recommended libraries and resources:

- **Milkdrop.co.uk Guides and Packs:** The site milkdrop.co.uk (no longer updated but still online in archived form) had an extensive *Beginner's Guide to MilkDrop preset writing* [104] and community-contributed tips. It's worth reading for a more tutorial-style introduction (explaining oscillators, common patterns, etc.). They also hosted preset packs and competitions.
- **Winamp Forums (Visualizations > MilkDrop Presets):** This is where authors like Rovastar, Flexi, Geiss, Eo.S., Martin, and many others exchanged presets. You can find themed packs (e.g., "Martins Big Mix", "Flexi's Pack") and "remixes" of presets. Remixing is common – one author might take another's preset and tweak it or combine with another idea, creating a new hybrid. This is totally encouraged in the scene (with credit given in the title usually). For someone starting out, looking at *simple* presets is educational. (Some famous presets are extremely complex; start with simpler ones to see cause and effect.)
- **ProjectM official preset pack:** ProjectM usually ships with a set of presets (often the Winamp default set plus many community ones). It's a grab-bag of good content. Also, on ProjectM's GitHub or SourceForge, they have additional preset packs (including experimental ones).
- **WACUP and Community Packs:** WACUP (WinAmp Community Update Project) is a modern fork of Winamp; their forums have a section for MilkDrop presets as well [105] [106]. Sometimes new packs or curated "best of" collections appear there or on Reddit.
- **Preset Mega Pack 2017/2020:** Some users have compiled all known presets into large zip files. For example, a user "fishbrain" on Reddit compiled about 52k presets from various sources. There's also mention of 15k presets converted for Butterchurn JSON format on GitHub [107].

Using presets from others is a great shortcut to visuals. You can literally load a preset and let it run. If you want to make it "yours," you can then tweak a color or add a small effect. That minimal change can sometimes produce a very different look, given how chaotic these systems are.

Automatic Generation and AI Experiments

The idea of creating MilkDrop presets automatically (by algorithmic means) is enticing, and there have been a few projects exploring this:

- **MilkDrop Mashup Tools:** The developer of MilkDrop3, "milkdrop2077", also created an open-source **preset generator/masher** tool [108]. This tool can randomly pick parts of presets and combine them, or randomly alter parameters, to produce new presets. Essentially it's like shuffling the DNA of existing visuals. Users have reported fun results by batch-generating presets this way and then selecting the best ones. MilkDrop3 itself incorporates some of this (the F9 double-preset creation is doing a form of mashup automatically) [94] [95]. The existence of an external tool suggests you can script preset generation outside of MilkDrop's UI – e.g., a program could parse .milk files, swap sections or tweak numbers, and save new .milk files. Because presets are text, this is very doable. Some have even used genetic algorithms (randomly mutate a preset, see if it "looks good", iterate) – though "looks good" is subjective, so usually a human has to pick or some proxy (like maximize motion or contrast) is used.

- **Machine Learning (MilkDropML/MilkDropLM):** Very recently, community members have started training language models on the corpus of presets. One project is **MilkDropLM**, a 7-billion-parameter model (and now a 32B version) fine-tuned to generate MilkDrop preset code [109] [110]. The

model was trained on over 10,000 high-quality presets and learned the "domain-specific language" of MilkDrop scripts [109]. The authors note that MilkDrop presets are a mix of mathematical expressions and unique syntax, which normal code models don't cover, so they specifically trained on this to get a model that understands how a `.milk` file is structured and what formulas create interesting effects [110] [111]. MilkDropLM can be prompted (via a description or just by asking for a preset) to output a new preset. The results still require curation – not every AI-generated preset will be amazing – but some have turned out impressively "artistic" and working. The model's goal is to *"empower users to create visuals without having to know how to write shaders from scratch"*, by letting AI handle the tricky coding [110]. For instance, a user might say "give me a preset with a pulsating star field that reacts to bass" and the model might produce a plausible .milk code for that. This is a cutting-edge approach and very cool for the MilkDrop community, which historically has hand-written everything. It effectively can remix the learned techniques of many authors and generate novel combinations.

- **Evolutionary/GA approaches:** Aside from mashups and AI, the concept of using a genetic algorithm to evolve presets was floated on forums and Hacker News. The idea is to parameterize a preset (or multiple presets) and then randomize values, use a fitness function (which could even be user preference: show 4 random presets and pick which one looks best, then breed from those). While I'm not aware of a polished end-user tool that does this, it's conceptually possible. One challenge is that "fitness" is hard to quantify algorithmically (what makes a visual good?); hence why interactive evolution with a human in the loop was considered. In any case, this approach is similar to how some visual artists evolve fractals or graphics by mutation – it could be done with MilkDrop if someone hooks into the parameters.

- **Image-based or procedural generation:** Some have tried using external data (like images or 3D models) to drive presets. MilkDrop itself doesn't support arbitrary image import (except as static textures in shaders, or via the "sprite" feature in some versions for small bitmaps). But one example: there's a pack called "MilkDrop Image-Based Presets" (from 2022) which presumably used some technique to incorporate images into presets (perhaps by encoding them in equations or using the texture feature cleverly) [112]. This is more of a hack/trick than a standard approach.

In summary, you have a spectrum from fully manual editing to fully automated generation:

- **If you want minimal effort**: use MilkDrop3's mashup generator (literally F9 + Space to get a random new visual) [95], or grab a random preset from a pack.
- **If you want some creative control but not heavy coding**: tweak existing presets (change colors, toggle waves/shapes, adjust a sine frequency or amplitude). Even a small tweak can produce a visibly distinct result, and this way you don't start from a blank slate.
- **For learning and coding from scratch**: use the guides and an interactive environment (Winamp or MilkDrop3). Start with simple goals like "make a circle that grows with the beat" – try coding that in a preset, etc. The learning curve is not too bad if you have math familiarity, and it's rewarding to see math formulas dance on music.
- **Leverage the community**: Don't reinvent the wheel – many effects (e.g., oscillating zoom, strobe on beat, spiral distortion) have known formula patterns in presets. You can borrow code snippets or whole sections from one preset to another. The result is often interesting – in fact many "remix" presets are exactly that: taking the per-pixel from one and the per-frame from another, for instance.

Starting Out: Recommended Utilities & Libraries

To get up and running quickly with making your own visualizations, here's a short list of open-source tools and resources:

- **ProjectM (libprojectM) and Example Apps** – *Library & Apps*. If you're a developer or just want a standalone visualizer, ProjectM is a solid choice. It's LGPL and runs on many platforms [113]. You can use the SDL standalone app to visualize any audio. It comes with a trove of presets to play with. While it lacks a built-in editor GUI, you can manually edit presets and drop them in the presets folder to test. This is great for integrating MilkDrop into your own software or using it in situations where Winamp isn't an option.
- **MilkDrop3 (Standalone Windows app)** – *Tool*. For Windows users, MilkDrop3 is highly recommended for creation. It's basically MilkDrop on steroids: backward compatible with all MilkDrop/ProjectM presets, and adds easier mixing, randomization, and debug tools [114] [95]. Because it can capture audio from anywhere, you can use it with modern streaming or DJ software.
- **Butterchurn (Web library)** – *Web Tool*. If you have basic web dev skills or just want a quick way to test presets, Butterchurn is excellent. It's open source (MIT license) and even available via NPM for embedding in web projects [115]. You can load thousands of presets converted to JSON (there's a repository with 15k presets in Butterchurn format [107]). For sharing your creations, Butterchurn is a convenient way (since anyone with a browser can see them, no install needed).
- **Preset Packs (Community libraries)** – *Content*. Grab a curated preset pack such as *"Cream of the Crop"* or others mentioned on forums [116]. These packs often highlight the best presets. Studying them or using them as a base will save you time. For instance, if you like how a preset reacts to bass, you can open it and see that the author perhaps used `zoom = 1 + 0.15*bass` in per-frame or something – then you can adapt that technique.

- **MilkDrop Preset Authoring Guide (Ryan's and the community one)** – *Documentation*. The official guide on Geisswerks (by Ryan Geiss) is a technical reference – very useful once you know basics, for looking up specific variables or functions [117]. The community "Beginner's Guide" (available via archive) is more tutorial-like. Having these on hand will answer a lot of "can I do X?" questions. For example, you might wonder "what does `meshx` variable do?" (It gives the mesh resolution user setting) [118] – the docs have these details.

- **Milkdrop2077 Preset Generator** – *Automation*. This is on GitHub and can automatically generate presets or mashups [108]. If you want to produce a bunch of random visuals to pick from, this could be fun. It's written in Pascal, but as an end-user you'd use the compiled tool – it might present a GUI or just spit out .milk files. (It's a niche tool for experimenters, but worth mentioning for "minimal user input" scenario.)

- **MilkDropLM AI Model** – *Experimental AI*. For the adventurous, the MilkDropLM model on HuggingFace can be tried in a Google Colab or using text-gen UI [119] [120]. You could input a prompt or just let it generate. It will output a preset in text form (often in a Markdown code block). You then copy that into a .milk file and test it. This requires some setup (and a decent GPU for the 7B or 32B model), so it's not the simplest path for everyone, but it's a cutting-edge way to get new presets. The model's creators have shared example outputs, and while not every AI preset is great, some are remarkably complex – showing that the model captured a lot of the "style" of presets [110].

Finally, one more tip: **when editing or generating presets, keep music running!** The visuals make the most sense and are easiest to tune when reacting to actual audio. If you're editing without music, you might misjudge speeds or intensities. A trick some use is to have a short loop or track that they know well and design the preset to that, ensuring beats and drops cause the desired effect. Then test on various genres (since a preset reacting well to, say, trance might look dull on classical music – some presets even include logic to detect beat patterns).

Conclusion: MilkDrop presets are a unique fusion of art and code – little reactive programs that create psychedelic imagery from sound. The `.milk` file format encodes this via sections for per-frame logic, per-pixel warping, custom shapes, and waves [1]. The engine executes these in a tight frame-by-frame loop, feeding in music frequencies and time, and outputting continuously evolving graphics [121]. Variables and math expressions flow through the pipeline, with special care for variable scoping and persistence to allow complex behavior across frames [68] [66]. MilkDrop 2 and 3 expanded the possibilities further with GPU shaders [76], higher object counts, and quality-of-life features, all while staying backward-compatible. Today, whether you use Winamp's classic plugin, ProjectM on your platform of choice, or MilkDrop3's modern enhancements, you have access to a vast community-created visual library and tools that can help you create your own audio-reactive visuals. With even AI entering the scene to suggest preset designs, the MilkDrop ecosystem remains vibrant and continues to evolve – much like the visuals it produces, it's an ever-changing, collaborative work of art and engineering. Enjoy experimenting, and soon you'll have your music **"dancing"** on screen with your very own preset!

Sources:

- MilkDrop Wiki/Encyclopedia: Preset composition and code types [1] [3] [53]
- MilkDrop Authoring Guide (Geisswerks): technical reference on per-frame, per-vertex, shapes, waves, variables, and shaders [18] [19] [38] [68]
- *An Introduction to projectM* (LWN.net, 2018): explanation of MilkDrop's equation system and ProjectM's implementation [121]
- MilkDrop3 (community project) documentation: features like double-presets, increased limits, ease-of-use additions [114] [95]
- MilkDrop3 VJ Union post: list of new capabilities (shapes/waves count, q1-64, debug mode, beat-trigger, etc.) [4] [122]
- ProjectM release notes: improvements to matching MilkDrop behavior (wave rendering, instanced shapes, shader translation) [92] [81]
- MilkDropLM (HuggingFace model card): description of AI model for generating presets [109] [110]
- MilkDrop forums and community posts: discussions of preset creation and tool links (e.g., mashup tutorials).

---

[1] [2] [3] [15] [53] [76] [104] [113] MilkDrop
https://en-academic.com/dic.nsf/enwiki/1065954

[4] [57] [82] [93] [94] [95] [96] [97] [98] [99] [114] [122] MilkDrop3 - VJ UNION
https://vjun.io/vdmo/milkdrop3-4hon

5 6 7 8 9 10 11 12 13 14 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 54 55 56 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 77 78 85 86 87 88 89 117 118 www.geisswerks.com
https://www.geisswerks.com/milkdrop/milkdrop_preset_authoring.html

58 79 90 121 An introduction to projectM [LWN.net]
https://lwn.net/Articles/750152/

80 81 83 84 91 92 Releases · projectM-visualizer/projectm · GitHub
https://github.com/projectM-visualizer/projectm/releases

100 Butterchurn Visualizer
https://butterchurnviz.com/

101 Butterchurn – A WebGL Implementation of the Milkdrop Visualizer
https://news.ycombinator.com/item?id=26813265

102 115 Butterchurn is a WebGL implementation of the Milkdrop Visualizer
https://github.com/jberg/butterchurn

103 73k+ Presets Megapack! - Winamp & Shoutcast Forums
https://forums.winamp.com/forum/visualizations/milkdrop/milkdrop-presets/4625022-73k-presets-megapack

105 r/milkdrop - Reddit
https://www.reddit.com/r/milkdrop/

106 My MilkDrop Presets Collection - WACUP
https://getwacup.com/community/index.php?topic=2080.0

107 ansorre/tens-of-thousands-milkdrop-presets-for-butterchurn - GitHub
https://github.com/ansorre/tens-of-thousands-milkdrop-presets-for-butterchurn

108 GitHub - milkdrop2077/milkdrop2077: MilkDrop2077 is a free and open-source presets generator / masher and randomizer for MilkDrop / projectM / BeatDrop Music Visualizer.
https://github.com/milkdrop2077/milkdrop2077

109 110 111 119 120 InferenceIllusionist/MilkDropLM-7b-v0.3 · Hugging Face
https://huggingface.co/InferenceIllusionist/MilkDropLM-7b-v0.3

112 MilkDrop Image-Based Presets Collection DEMO2 (2022) - YouTube
https://www.youtube.com/watch?v=-40_a-E3gRg

116 NestDrop: Presets Collection – Cream of the Crop | The Fulldome Blog
https://thefulldomeblog.wordpress.com/2020/02/21/nestdrop-presets-collection-cream-of-the-crop/